

Activity Recognition Using IMU and SVM – Embedded Inference on a Microcontroller

Aristide-Herve Dossou

*Corresponding Author: aristide-herve.dossou9@etu.univ-lorraine.fr

Abstract

This work presents the design and implementation of an embedded activity-recognition system based on an inertial measurement unit (IMU) and a Support Vector Machine (SVM) classifier. Three activities are targeted: rest, left-right motion, and up-down motion. The workflow includes dataset acquisition, signal preprocessing, training of an RBF-kernel SVM in Google Colab, exporting the trained model to MicroPython, and real-time inference on an STM32 microcontroller. The study emphasizes lightweight feature-extraction methods adapted to low-power hardware and discusses the balance between accuracy, computation, and memory usage when deploying machine-learning models on microcontrollers. Results show that reliable gesture recognition can still be achieved under strict resource constraints, demonstrating the relevance of classical ML models such as SVMs for embedded human-machine interaction.

1 Introduction

IMU-based activity recognition is widely used in gesture interfaces, wearable devices, and robotics. Running machine-learning algorithms on a microcontroller, however, requires strict efficiency in memory usage and computation.

In this project, we implement a lightweight processing pipeline: extract statistical features from IMU windows and classify gestures using an RBF SVM exported as a JSON model executed in MicroPython. The objective is to achieve real-time gesture recognition with limited resources.

2 Materials and Methods

2.1 Hardware

The experiments were conducted using the following embedded platform:

- NUCLEO-WB55 development board (STM32WB55 MCU).
- X-NUCLEO IMU shield, providing a 3-axis accelerometer and gyroscope.
- USB cable for powering the board and transferring files.

This hardware setup allows on-board acquisition of inertial measurements and real-time execution of the MicroPython inference pipeline.

2.2 Software

The software environment required for the project includes:

- **PuTTY** (or an equivalent serial terminal emulator) configured at *115200 baud, 8N1* for real-time monitoring and data collection.
- **MicroPython** running on the NUCLEO board, with custom scripts provided by the instructors (`main.py`, `moduleimu.py`, `svm_micro.py`).
- **Google Colab**, used for data preprocessing, visualization, and model training with: TensorFlow/Keras, scikit-learn pandas, matplotlib.

A provided **Jupyter notebook** (`SVM_imu.ipynb`) used to train the RBF SVM classifier and export the resulting model.

2.3 Dataset Acquisition

Three classes of hand motion were recorded using the IMU: **Rest**, **Left-right motion** and **Up-down motion**. For each class, approximately **60 s** of data were collected. Before each acquisition, the output CSV file name was edited directly in `main.py`. The IMU streamed samples at a nearly constant sampling period of approximately 0.6s, stored in the following format: t, ax, ay, az, gx, gy, gz. Quality checks were performed to ensure absence of sensor saturation, correct dynamic range for each class.

2.4 Data Loading, Preprocessing and Windowing

A dedicated loader function, `load_csv_imu_time_first`, is used to retain only the acceleration components from the dataset: ax, ay, az. The resulting IMU data is returned as a NumPy array with the shape: `(n_samples=1000, 3)` after the cleaning. Where `n_samples` represents the total number of samples in the dataset, and the second dimension corresponds to the three acceleration components (ax, ay, az). The raw signals were segmented into overlapping temporal windows. To transform the continuous IMU recordings into fixed-size learning samples, the dataset is segmented using a sliding-window approach. Each window corresponds to a sequence of consecutive samples of length **W (window size of 100)**, and windows are extracted for every **S samples (stride of 10)**. Practically, for each class signal, the algorithm iterates through the time series with a step of S, selects the subsequence `X[i : i + W]`, and stores it as a training instance. This procedure produces a structured tensor of shape `(N_windows, W, n_features)`, where `n_features` is the number of IMU axes retained for learning. Windowing is essential because IMU recordings have variable duration, whereas machine-learning models require inputs of constant dimensionality. By converting raw time series into uniform segments, the method

enables consistent training, preserves local temporal dynamics, and increases the number of usable samples through controlled overlap. This results in 273 extracted windows.

$$N_{\text{windows}} = \left\lfloor \frac{n - \text{window_size}}{\text{stride}} \right\rfloor + 1$$

For each set of motion we have recorded 1000

$$\text{samples: } N_{\text{windows}} = \left\lfloor \frac{1000 - 100}{10} \right\rfloor + 1 = 91.$$

This results in the total of $91 \times 3 = 273$ windows. Each window has a length of **100** samples and **3** channels (ax, ay, az)

2.5 Training, Validation, and Test Protocol

The supervised learning stage relies on a classical split between training and test data. After the windowing step, all segmented samples and their associated class labels are partitioned using the `train_test_split` function from scikit-learn with the argument `stratify=yw`. This operation produces two datasets: Training set: used to fit the feature scaler (standardization) and to train the SVM classifier. Test set: kept strictly isolated during training and used exclusively for final performance evaluation. The key parameter controlling this partition is `test_size`, which specifies the proportion of samples assigned to the test set. We used a **test size of 0.2**, providing a good balance (80/20) between the amount of data available for model learning and the amount reserved for reliable evaluation. (Hastie et al., 2009) We therefore had 218 windows for the training and 55 for the test. The other key training parameters defined in the notebook are as follows:

- **RANDOM_STATE = 42**: Ensures reproducibility of data splitting and random operations.
- **EPOCHS = 100**: Maximum number of epochs for neural network training (though the final model is an SVM, this parameter remains defined).
- **BATCH = 64**: Batch size used during neural network training.
- **VAL_SPLIT = 0.2**: Fraction of the training set reserved for validation in Keras models.

2.6 Statistical Features and SVM Classification

To reduce the dimensionality of the IMU time series while retaining their discriminative structure, each window is transformed into a set of 12 statistical features. These features fall into two categories: per-axis statistics and cross-axis correlation measures.

2.6.1 Per-axis statistics (9 features)

For each accelerometer axis $x \in \{a_x, a_y, a_z\}$

- **Mean value:**

$$\mu_x = \frac{1}{T} \sum_{t=1}^T x_t$$

- **Standard deviation:**

$$\sigma_x = \sqrt{\frac{1}{T} \sum_{t=1}^T (x_t - \mu_x)^2}$$

- **Signal energy (mean-square value):**

$$E_x = \frac{1}{T} \sum_{t=1}^T x_t^2$$

Since these three statistics are computed independently for the three axes, this yields a total of 9 features.

2.6.2 Cross-axis correlations (3 features)

Defined using covariance divided by product of stdevs: $\rho_{xy} = \frac{\sigma_x \sigma_y}{\text{cov}(x,y)}$

Computed for:

- ρ_{xy}
- ρ_{xz}
- ρ_{yz}

Total: 12 features per window. After feature extraction, each window is represented as a 12-dimensional feature vector. For the training split, this results in a matrix of the form: $\mathbb{R}^{(N_{\text{train}}, 12)}$. If raw IMU windows were used directly as SVM inputs, the dimensionality would be substantially

higher. With a window size $W = 100$ samples and three axes, each window would contain:

$$W \times 3 = 100 \times 3 = 300 \text{ values}$$

Such high-dimensional vectors would increase the computational cost of training and inference. The classification stage uses a Support Vector Machine preceded by feature standardization. The key parameters of the SVM classifier are the following:

- **Kernel: RBF** — allows nonlinear decision boundaries suitable for human-motion signals.
- **C = 3.0** — controls regularization; moderately high to balance margin width and classification accuracy.
- **Gamma = scale** — automatically set to

$$\gamma = \frac{1}{n_{\text{features}} \cdot \text{Var}(X)},$$

which provides a robust, data-adaptive default.

- **class_weight = balanced** — compensates for unequal numbers of windows in each class, improving robustness.

This configuration is consistent with common recommendations in the literature for IMU-based gesture and activity recognition, where class overlap and nonlinear separability are frequently observed.

2.7 Data Exploration and Visualization: Contribution of UMAP

In this study, UMAP (Uniform Manifold Approximation and Projection) was used to reduce the dimensionality of the data and enable visualization of the samples in a two-dimensional space. The goal is to better understand the underlying structure of the data and visualize the separation between different activity classes.

UMAP is a nonlinear dimensionality reduction technique that is particularly effective for visualizing complex data while preserving both local

and global relationships. Unlike methods such as PCA (Principal Component Analysis), UMAP is capable of capturing nonlinear structures, which is crucial when the data is complex and the separation between classes is not simply linear.

2.7.1 Contribution of UMAP in this Study

The primary advantage of using UMAP here is its ability to project data from a high-dimensional space (with a large number of features per sample) into a two-dimensional space, while maintaining the topological structure of the data. This allows us to:

- **Visualize the structure of the data:** By reducing the dimensionality of the training and test data (derived from IMU sequences), we can observe how the different activity classes are distributed and separated in a 2D plot.
- **Detect anomalies or overlaps:** If some classes overlap heavily, this may indicate ambiguities or similarities in the characteristics of the sequences, which could be further explored.

3 Results

Training results

We applied UMAP to both the training and test data and generate 2D projections for visualization. We have got the following results:

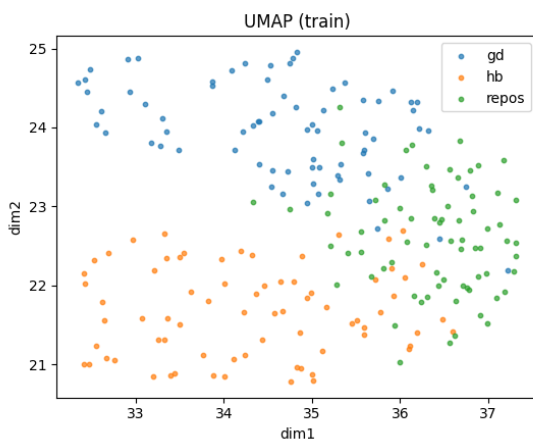


Figure 1: UMAP (Train)

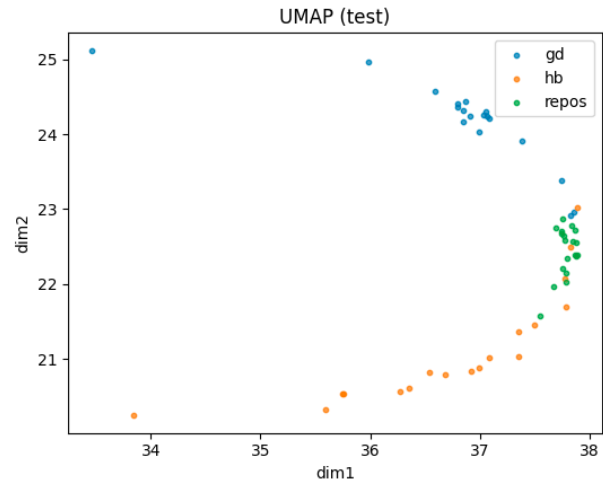


Figure 2: UMAP (Test)

By observing the plots generated, we can see that the points corresponding to each class are well separated. Only a few points of the different regions overlap. The clear separation between the classes suggests that the features are good at distinguishing between them. UMAP allowed us to identify the 3 groups or clusters.

We can also see that the separation between classes is consistent across both training and new test data, which indicates how well the model generalizes. To improve the separation between clusters, we applied a sliding window of 5 to smooth the data. This preprocessing step was performed to reduce the noise and make the underlying patterns more apparent, which can improve the performance of the subsequent model. After applying the sliding window average, we visualized the resulting data using UMAP.

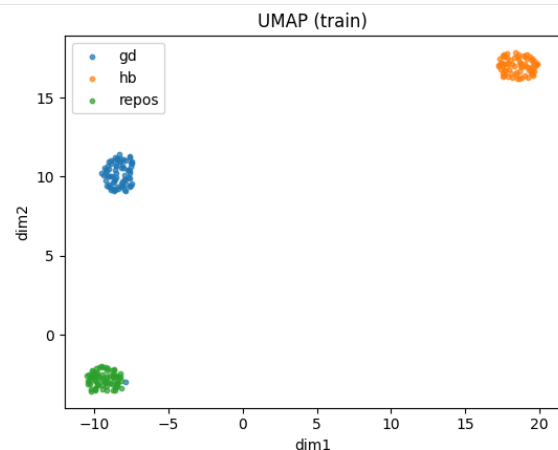


Figure 3: UMAP (Train)

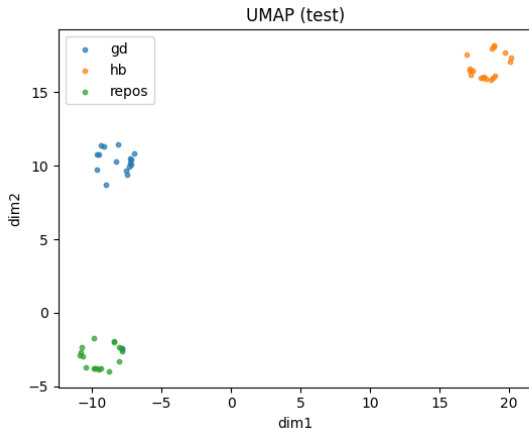


Figure 4: UMAP (Test)

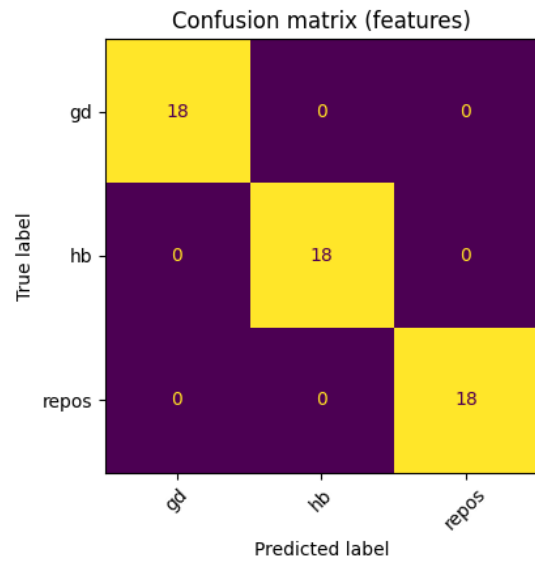


Figure 5: Confusion Matrix of the SVM

The impact of this preprocessing step is evident in the clustering results. Specifically, after applying the sliding window average, the clusters of different activity classes became more distinctly separated. This improvement in separation suggests that the sliding window average helped reveal more structured and meaningful features in the data, which likely contributed to a more robust and accurate model.

3.1 SVM Classification Results

The Support Vector Machine classifier achieved excellent performance on the statistical-feature dataset derived from the IMU windows. After feature extraction and standardization, the RBF-kernel SVM was trained on the preprocessed training set and evaluated on an independent test set containing 54 windows (18 per class).

The classification report shows perfect performance across all three activities—left-right motion, up-down motion, and rest motion—with precision, recall, and F1-score all equal to 1.00. This means that the model correctly classified every test sample without any false positives or false negatives. The confusion matrix also confirms a perfect separation between the three classes:

3.2 Embedded Measurements and Validation

To assess the real-time performance and robustness of the embedded activity-recognition system, several on-device validation experiments were conducted. These experiments complement the evaluation performed during SVM training and provide insights into the system's practical behavior when deployed on the STM32 microcontroller.

- Inference Latency

The prediction time for a single window was measured directly on the microcontroller using a hardware timer. The value of the latency is **16ms**.

- Robustness Across Gesture Speeds

Each gesture class (rest, left-right motion, up-down motion) was repeated under three conditions: Slow execution, Normal execution, Fast execution. The system's predicted labels were observed on the serial output in each case. Robust performance is reflected by consistent predictions regardless of the speed of motion, demonstrating that the statistical features remain discriminative across motion dynamics. The sampling frequency was also varied (50 Hz, 100 Hz, 200 Hz) to investigate the influence of acquisition rate on classification reliability.

- On-Device Confusion Matrix

To quantify classification accuracy directly on the microcontroller, a “mini” confusion matrix was constructed from controlled gesture trials. For each class, a fixed number of gestures (e.g., 10 repetitions) were performed, and the system’s displayed predictions were manually tallied.

For a given gesture type (e.g., left–right), the number of predictions falling into each output class (rest, left–right, up–down) was counted.

Table 1: Mini confusion matrix obtained from on-device gesture validation.

True class	Rest	Left/Right	Up/Down
Rest	10	0	0
Left/Right	0	10	0
Up/Down	0	0	10

The resulting table summarizes the empirical on-device performance and reveals an excellent correct recognition rate.

4 Discussion

The experimental results highlight that simple statistical features extracted from short IMU windows are highly effective for discriminating between the three target activities. The SVM classifier with an RBF kernel achieved perfect accuracy on the test set, supported by a confusion matrix with no misclassifications.

An important observation is the positive impact of applying a moving-average filter prior to feature extraction. During development, a sliding mean with a window of four samples was applied both during preprocessing and later on the embedded device. This lightweight smoothing step significantly reduced high-frequency noise and sensor jitter, leading to much cleaner window statistics and more stable feature distributions. As a result, the classifier exhibited strong robustness to noise, even under conditions where the raw accelerometer readings showed noticeable fluctuations.

Despite the excellent performance, the current dataset likely benefits from controlled acquisition conditions—consistent gestures, stable device placement. This reduces intra-class variability and contributes to near-ideal separability. Expanding the dataset to include different sensor placements, or more subtle gesture variations would provide a more realistic measure of generalization. In such cases, changing some features or more advanced deep-learning embeddings may become necessary. Nonetheless, the current SVM-based approach remains extremely efficient for some applications.

Conclusion

This study presents a complete lightweight activity-recognition pipeline combining IMU sensing, moving-average filtering, sliding-window segmentation, handcrafted statistical features, and an RBF-SVM classifier deployable on microcontrollers. The system successfully identifies three distinct gestures with perfect test accuracy, demonstrating that carefully selected features and modest preprocessing can deliver high performance on embedded hardware. A key contribution of this work is showing that even a very small smoothing filter (window size of 5) provides excellent noise immunity, improving stability both during training and on-device inference. This finding reinforces the relevance of simple signal-processing techniques in embedded machine-learning workflows, where computational constraints are strict.

Future work may explore the addition of more gestures and varying test conditions. Yet, the current results confirm that low-power gesture recognition can be achieved with minimal computational overhead while maintaining outstanding accuracy and robustness.

References

- T. Hastie, R. Tibshirani, and J. Friedman. The elements of statistical learning: Data mining, inference, and prediction. 2nd ed., springer series in statistics, 2009. *Springer Series in Statistics*, page 745, 2009.